Praktikum 1 zu Parallele Programmierung

Tim Heldmann, Lukas Rothenberger



Wintersemester 2026/2027
03.11.2026

Allgemeines

- Die Standardsprache im Quellcode ist Englisch, weswegen auch die Typen/Methoden so benannt sind.
- Es gibt Fußnoten, wenn Konzepte das erste Mal verwendet werden; dort können Sie sich weiter informieren.
- *Es gibt Hinweise (zu gutem Programmierstil oder anderem) in kursiver Schrift. Diese werden nicht bewertet.*
- Stellen Sie sicher, dass Ihre Abgabe auf dem Hochleistungsrechner der TU Darmstadt lauffähig ist, andernfalls ist mit Punktabzug zu rechnen.

Bepunktung der Aufgaben

- Bei jeder Teilaufgabe stehen die erreichbaren Punkte dabei, welche die Anzahl an funktionalen Tests für die jeweige Teilaufgabe angibt.
- Für jeden funktionalen Test gilt: Sollte der funktionale Test selbst, oder der zugehörige öffentliche Test auf kompilierbarkeit des Programmes fehlschlagen, nicht kompilieren oder länger als eine Minute brauchen, erhalten Sie keinen Punkt dafür. Sollte der Test innerhalb von einer Minute korrekt durchlaufen, erhalten Sie dafür einen Punkt.
- Sie erhalten keine Punkte für Teilaufgaben mit fehlschlagenden öffentlichen Compiletests! Sie sind dafür Verantwortlich, einen kompilierenden, mit den Vorgaben kompatiblen Code einzureichen. Verändern Sie die vorgebenen Signaturen nicht!

Zusätzliche Hinweise

- Nennen sie **keine** Ihrer Dinge (z.B. Funktionen, Dateien, Klassen) mit `ppws26` als Bestandteil vom Namen.
- Ihnen steht frei weitere Funktionalität zu implementieren, solange nicht explizit anders geregelt.
- Achen Sie darauf, dass Sie nur standardkonformes C++ benutzen. Das Benutzen von compiler- / laufzeit- / betriebssystemspezifischen Erweiterungen und Funktionalitäten kann zu Punktabzug führen. Explizit erlaubt sind die von uns vorgegebenen Dinge (Frameworks für die funktionalen Tests und die Laufzeittests, Benutzung von Typen wie `std::uint32_t` etc.)
- Beachten Sie, dass die Tutor:innen deren Sprechstunden Sie besuchen nicht notwendigerweise Ihre Abgaben bewerten.

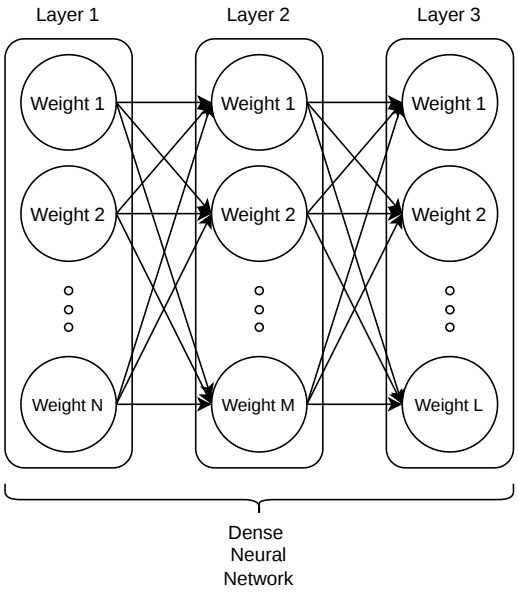
Allgemeine Hinweise
Wenn eine Methode einen Parameter nicht ändert, sollten Sie den als `const` *klassifizieren – dies vermeidet Programmierfehler.*[1]
Wenn eine Methode das aktuelle Objekt nicht ändert, sollten Sie diese als `const` *klassifizieren. Dadurch sind die Methoden aufrufbar, auch wenn das Objekt als solches selbst* `const` *ist (siehe vorherigen Hinweis).*[2]
Wenn eine Methode keine Exception werfen kann, sollten Sie diese als `noexcept` *markieren.*[3]
Wenn eine Methode einen Wert zurück gibt und ein logischer Fehler wäre, diesen nicht zu speichern, können Sie diese mit `[[nodiscard]]` *markieren. Wird der Wert nicht beim Aufruf gespeichert, erzeugt dies eine Warnung.*[4]

[1] `https://en.cppreference.com/w/cpp/language/cv`
[2] `https://en.cppreference.com/w/cpp/language/member_functions`
[3] `https://en.cppreference.com/w/cpp/language/noexcept_spec`
[4] `https://en.cppreference.com/w/cpp/language/attributes/nodiscard`

Aufgabe 1: Einleitung

Im Rahmen dieses Praktikums werden Sie eine Bibliothek zum Erstellen und Trainieren von neuronalen Netzwerken programmieren. Das Ziel ist es, vertrauter mit C/C++ zu werden. Am Ende dieses Praktikums sollten Sie die Unterschiede zwischen Header- und Sourcedateien erläutern und erklären können, welche Klassen und Funktionen aus welchen Gründen wo zu finden sind. Sie sollten grundlegende Konzepte der Vererbung und Polymorphie erkennen, erklären und erweitern können. Sie sollten außerdem die Unterschiede zwischen einem Programm, einer Bibliothek bzw. Executables und Shared Objects erklären können. Viele der grundlegenden Programmierkonzepte sollten Ihnen aus FOP und AUD bekannt sein. Die Grundlagen von neuronalen Netzwerken haben Sie in EKI gelernt. Es geht hier explizit um die Übertragung dieser Konzepte auf C/C++.[5] Strukturell besteht der Code aus zwei Teilen: einer Bibliothek zum Erstellen und Trainieren von Dense Neural Networks (DNN) sowie einem Client-Programm, das die Bibliothek benutzt, um eine Machine-Learning-Aufgabe zu erfüllen.

Das Netzwerk und seine Layer werden explizit modelliert. Das Netzwerk verwaltet die Inputs, die Layer und die Outputs sowie die Loss-Function zur Bestimmung der aktuellen Güte des Netzwerks. Jedes Layer verwaltet seine Weights, Biases und seine Activation Function. Während des Trainings wertet das Netzwerk jedes Layer vorwärts aus (Forward Propagation), berechnet die Loss-Function und geht anschließend rückwärts durch jedes Layer (Backward Propagation), sodass jedes Layer seine Weights und Biases anpassen kann.



Die Aufgaben führen Sie stückweise in einem Bottom-up-Ansatz durch die Erstellung der Bibliothek.

Laden Sie sich den Code aus Moodle herunter und öffnen Sie ihn in einer Integrierten Entwicklungsumgebung (IDE) Ihrer Wahl. Stellen Sie sicher, dass Ihre IDE CMake unterstützt und als Compiler GCC-13 verwendet.

Ihr Code muss wie im Folgenden beschrieben auf dem Lichtenberg-Cluster kompilierbar und ausführbar sein, um bewertet zu werden.

Wir empfehlen außerdem dringend, eine Source-Code-Versionierungssoftware wie Git zu benutzen, da dies die Kollaboration erleichtert und fehlerhafte Änderungen schneller rückgängig gemacht werden können.

[5] Sollten Sie mit den Algorithmen hinter neuronalen Netzwerken nicht vertraut sein, empfehlen wir Ihnen sich darin einzuarbeiten. Artikel wie dieser: `https://medium.com/data-science/lets-code-a-neural-network-in-plain-numpy-ae7e74410795` können ein guter Startpunkt sein.

Aufgabe 2: Kompilieren, Ausführen und Testen

Der folgende Abschnitt beschreibt das Kompilieren, Ausführen und Testen des Codes und bezieht sich auf den Lichtenbergcluster. Sie können grundsätzlich alle Schritte auf einem lokalen Linux-System ausführen, beachten Sie jedoch, dass Sie bei der Einrichtung u. U. auf sich alleine gestellt sind, da wir Sie aus Kapazitätsgründen nur bei Problemen auf bzw. mit dem Cluster unterstützen können. Testen Sie Ihren Code unbedingt vor der Abgabe auf dem Cluster, da wir keine Fehler während der Kompilierung o. Ä. für Sie beheben werden können.

- Laden Sie die Module GCC 13 sowie CMake 4.23 mit Hilfe von:

```
module load gcc/13 cmake/4.2
```

- Vorbereitung (Beginnend im Verzeichnis des Praktikums):

```
wget https://data.pjreddie.com/files/mnist_test.csv
```

- Ausführen der Signatur- und Compiletests

```
tests/scripts/test_compilation.sh
```

- Ausführen der Bewertungstests

```
tests/scripts/run_unit_tests.sh
```

- Kompilieren des Programms:

```
mkdir build
cmake -B build
make -C build -j
```

- Ausführen des Programms auf dem Loginknoten (zu Testzwecken):

```
build/DNN_app
```

- Ausführen des Programms auf einem Rechenknoten[6]:

```
Anpassen von job.sh (z.B. ersetzen von TUID)
sbatch job.sh       // Submittieren des Jobs
squeue              // Abfragen es Ausführungsstatus
```

  Beachten Sie, dass das Codeskelett nicht in unbearbeiteter Form compiliert. Ihr code sollte nach vollständiger Bearbietung der Aufgabe 3a) compilen.

[6] Weitere Informationen zum Batchsystem finden Sie im folgenden Abschnitt "Hinweise zum Lichtenberg“ sowie auf den Seiten des HRZ: `https://www.hrz.tu-darmstadt.de/hlr/nutzung_hlr/rechenjobs_und_batchsystem_hlr/index.de.jsp`

Aufgabe 3: Activation functions (Gesamt: 2 Punkte)

Machen Sie sich zunächst mit der Implementierung der Klasse `ActivationFunction` in `include/lib/activation/ActivationFunction` vertraut.

3a) Header-Dateien (1 Punkt)

Vervollständigen Sie die im folgenden benötigten Header-Dateien

- `include/lib/activation/LeakyReLU.h`
- `include/lib/activation/ReLU.h`
- `include/lib/activation/ScaledSigmoid.h`
- `include/lib/activation/Sigmoid.h`

wobei folgendes gelten soll:

- Jede Aktivierungsfunktion ist als eigene Klasse im namespace `dnn` zu deklarieren
- Jede Aktivierungsfunktion erbt von `ActivationFunction`
- Jede Aktivierungsfunktion überschreibt alle geerbten Funktionen
- `LeakyReLU`, `ReLU`, sowie `Sigmoid` verwenden die `default` Konstruktoren und Destruktoren
- Verändern Sie die Signatur des von `ScaledSigmoid` definierten Konstruktors, die Klassenattribute `min` und `max`, sowie die Funktionen `getMin()` und `getMax()` nicht. Geerbte Funktionen sollen wie zuvor überschrieben werden.
- Die genannten Header-Dateien definieren keine Funktionen, sondern deklarieren diese lediglich.

3b) Implementierung (1 Punkt)

Implementieren Sie die zuvor deklarierten Aktivierungsfunktionen sowie Konstruktoren gemäß der folgenden vorgaben in `src/lib/activation/LeakyReLU.cpp`, `ReLu.cpp`, `ScaledSigmoid.cpp`, und `Sigmoid.cpp`.
Nutzen Sie `std::exp`[7].

- ReLU:

$$activation(x) = \begin{cases} x & \text{wenn } x > 0 \\ 0.0 & \text{sonst} \end{cases}$$

$$activationBackwards(x) = \begin{cases} 1.0 & \text{wenn } x > 0 \\ 0.0 & \text{sonst} \end{cases}$$

$$getName() := relu$$

- LeakyReLU:

$$activation(x) = \begin{cases} x & \text{wenn } x > 0 \\ 0.01 \cdot x & \text{sonst} \end{cases}$$

$$activationBackwards(x) = \begin{cases} 1.0 & \text{wenn } x > 0 \\ 0.01 & \text{sonst} \end{cases}$$

$$getName() := leaky_relu$$

[7] `https://en.cppreference.com/cpp/numeric/math/exp`

- Sigmoid

$$activation(x) = \frac{1.0}{1.0 + e^{-x}}$$
$$activationBackwards(x) = activation(x) \cdot (1 - activation(x))$$
$$getName() = sigmoid$$

- ScaledSigmoid

$$activation(x) = min + (max - min) \cdot \frac{1}{1 + e^{-x}}$$
$$activationBackwards(x) = (max - min) \cdot \frac{1}{1 + e^{-x}} \cdot (1 - \frac{1}{1 + e^{-x}})$$
$$getName() = scaled_sigmoid$$
$$getMax() := max$$
$$getMin() := min$$

Aufgabe 4: Math utils (Gesamt: 2 Punkte)

Im Allgemeinen ist es hilfreich Funktionalität, die nicht Teil einer fest definierten Aufgabe sind, aber an mehreren Stellen im Programm hilfreich sein kann in eine eigene Datei auszulagern. Wir legen deswegen eine `MathUtils` Datei an, in der nützliche mathematische Funktionen definiert werden.

4a) Skalarprodukt und Summe (1 Punkt)

Implementieren Sie die in `include/lib/MathUtils.h` deklarierten Funktionen `dotProduct` und `sum` in `src/lib/MathUtils.cpp`. Verändern Sie die Signaturen der Funktionen nicht. `dotProduct` soll das Skalarprodukt der Vektoren `a` und `b` berechnen, wohingegen `sum` die Summe aller Elemente im Vektor `values` ermittelt.

4b) Normalisieren (1 Punkt)

Implementieren Sie die folgenden beiden Templatefunktionen mit Typparameter `T` als Teil des Namespace `dnn` in `include/lib/MathUtils.h`:

- `std::vector<double> normalize(const std::vector<T>& values);`
- `std::vector<double> normalize(const std::vector<T>& values, int maxValue);`

Beide Funktionen sollen den übergebenen Vektor `values` in einen normalisierten Vektor übersetzen und diesen zurückgeben.
`std::vector<double> normalize(const std::vector<T>& values)` soll die Normalisierung vornehmen, indem jedes Element von `values` durch das größte Element in `values` geteilt wird.
`std::vector<double> normalize(const std::vector<T>& values, int maxValue)` hingegen soll die Normalisierung durch dividieren durch `maxValue` berechnen.

Aufgabe 5: Layers (Gesamt: 4 Punkte)

Sie finden die Deklaration der Klasse `Layer` in `include/lib/Layer.h`. Ziel dieser Aufgabe wird es sein, die deklarierten Methoden zu implementieren. Verändern Sie die vorgegebenen Signaturen nicht.

5a) Forward propagation (1 Punkt)

Implementieren Sie die Methode `forwardPropagation` der Klasse `Layer` in `src/lib/Layer.cpp`. Initialisieren Sie zunächst `preActivationOutput` sowie `activationOutput` so, dass beide `layerOutputSize` viele Elemente beinhalten. Die Elemente der Vektoren sollen mit `0.0` initialisiert werden. Berechnen Sie die forward Propagation wie folgt:

$$preActivationOutput[i] = weights[i] \cdot previousLayerOutput + bias[i]$$

$$activationOutput[i] = activation(preActivationOutput[i])$$

5b) Backward propagation (1 Punkt)

Implementieren Sie die Methode `backwardPropagation` der Klasse `Layer` in `src/lib/Layer.cpp` gemäß der folgenden Berechnungsvorschriften.
Initialisieren Sie zunächst einen `std::vector<double> dPreActivation` der Länge `layerOutputSize` wie folgt:

$$dPreActivation[i] = dActivationOutput[i] \cdot activationBackwards(preActivationOutput[i])$$

Setzen Sie dann die Größe von `dWeights` so, dass eine `layerOutputSize`$\times$`layerInputSize` große Matrix von Gewichten dargestellt werden kann. Die Gewichte sollen wie folgt berechnet werden:

$$dWeights[i][j] = dPreActivation[i] \cdot previousLayerOutput[j]$$

Sichern Sie die berechnete `dPreActivion` in `dBias`.
Aktualisieren sie `accumulatedDWeights` sowie `accumulatedDBias` wie folgt:

$$accumulatedDWeights = accumulatedDWeights + dWeights$$

$$accumulatedDBias = accumulatedDBias + dBias$$

Inkrementieren Sie `numProcessedSamples`.
Setzen Sie die Länge von `dPreviousLayerOutput` auf `layerInputSize` und initialisieren Sie die enthaltenten Elemente mit `0.0`. Berechnen Sie für $j \in 0 \dots layerInputSize - 1$:

$$dPreviousLayerOutput[j] = weightsColumn \cdot dPreActivation$$

wobei für jedes j `weightsColumn` wie folgt berechnet wird:

$$weightsColumn[i] = weights[i][j]$$

5c) Update parameters (1 Punkt)

Implementieren Sie die Methode `updateParameters` der Klasse `Layer` in `src/lib/Layer.cpp`, welche `weights` und `bias` gemäß der folgenden Berechnungsvorschriften aktualisiert:

$$weights = weights - learningRate \cdot dWeights$$

$$bias = bias - learningRate \cdot dBias$$

5d) Commit to weights (1 Punkt)

Implementieren Sie die Methode `commitToWeights` der Klasse `Layer` in `src/lib/Layer.cpp`, welche die Gewichte mit Hilfe von `updateParameters` aktualisieren soll. Bereiten Sie zunächst `accumulatedDWeights` sowie `accumulatedDBias` wie folgt vor:

$$accumulatedDWeight = accumulatedDWeights/numProcessedSamples$$

$$accumulatedDBias = accumulatedDBias/numProcessedSamples$$

Aktualisieren Sie anschließend die Gewichte in `weights` sowie `bias`.
Setzen Sie alle Elemente von `accumulatedDWeights` und `accumulatedDBias` auf `0.0`, sowie `numProcessedSamples` auf `0`, um die nächste Iteration vorzubereiten.

Aufgabe 6: Network (Gesamt: 4 Punkte)

Ziel dieser Aufgabe wird es sein die bereitgestellte Implementierung des Netzwerks zu vervollständigen. Implementieren Sie alle im folgenden geforderten Methoden der Klasse `Network`, deklariert in `include/lib/Network.h`, in `src/lib/Network.cpp`.

6a) Constructor and Destructor (1 Punkt)

Implementieren Sie zunächst den Konstruktor der Klasse `Network`.
Zum Einen müssen Sie dazu die mittels `lossFunction` übergebene Funktion in `ownedLossFunction` sichern, damit diese im folgeden vom Netzwerk weiterverwendet werden kann.

Zum Anderen müssen die durch `layerParameters` beschriebenen Layers erzeugt und im Netzwerk registriert werden. Das erste Element von `layerParameters` stellt dabei die zu verwendenden Gewichte dar, das zweite Element den zu verwendenden Bias, und das dritte Element beschreibt die jeweils zu verwendende Aktivierungsfunktion des Layers. Eingabe- und Ausgabegrößen der Layer lassen sich jeweils durch die Länge von `weights[0]` bzw. `weights` selbst bestimmen.

Implementiernen Sie weiterhin den Destruktor der Klasse `Network`.

6b) addLayer und getLayerInput (1 Punkt)

Implementieren Sie die Methoden `addLayer` sowie `getLayerInput`.
Dabei soll `addLayer` die Möglichkeit bieten, dem Netzwerk neues Layer mit zufälligen Gewichten aber festgelegten Dimensionen sowie festgelegter Aktivierungsfunktion hinzuzufügen. Die Parameter `inputDim` und `outputDim` beschreiben Eingabe- sowie Ausgabegrößen des Layers. Der Parameter `activationFunction` beinhaltet die zu verwendende Aktivierungsfunktion. Um reproduzierbarkeit der Ergebnisse zu ermöglichen, soll zudem zum erzeugen der zufälligen Gewichte der in `nextSeed` enthaltene Seed verwendet werden. Inkrementieren Sie `nextSeed` nach dem erzeugen jedes Layers.
Die Methode `getLayerInput` soll den erhaltenen Input des Layers an Stelle `layerId` zurückgeben. Der erhaltene Input des Layers entspricht dabei dem aus Aufgabenteil 4a bekannten `activationOutput` des vorherigen Layers oder `initialInput`, falls kein solches existiert.

6c) fullForwardPropagation and fullBackwardPropagation (1 Punkt)

Implementieren Sie die beiden Methoden `fullForwardPropagation` sowie `fullBackwardPropagation`.

Ausgehend von `input` als Aktivierung für Layer 0, soll `fullForwardPropagation` die Ausgabe jedes Layers berechnen, wobei die berechneten Ausgaben eines Layers jeweils die Eingabe des nachfolgenden darstellen. Die berechnete Ausgabe des letzten Layers, und somit des Netzwerks, ist in `output` abzulegen.

Ähnlich soll `fullBackwardPropagation` zunächst auf Basis des `predictedOutput` sowie des `actualOutput` des Netzwerks den Gradienten `std::vector<double> dACurr` bestimmen. Nutzen Sie dazu die Methode `getGradient`, welche von der im Netzwerk hinterlegten Loss-Funktion bereitgestellt, und in der folgenden Aufgabe von ihnen Implementiert wird. Nach bestimmen des initialen Gradienten `dACurr` soll dieser verwendet werden, um die Gewichte des letzten Layers des Netzwerks mit Hilfe der zuvor implementierten `backwardPropagation` anzupassen. Nach dieser Anpassung wird `dACurr` durch die Ausgabe des vorherigen Layers ersetzt und der beschriebene Vorgang wiederholt, bis auch die Gewichte des ersten Layers des Netzwerks angepasst wurden.
Hinweis: Bei Bearbeitung in Reihenfolge der Aufgaben ist `getGradient` zum aktuellen Zeitpunkt noch nicht implementiert.

6d) trainBatch und computeAccuracy (1 Punkt)

Implementieren Sie die beiden Methoden `trainBatch` und `computeAccuracy`.

Die Methode `trainBatch` prüft erst ob eine Loss-Function existiert und danach ob die inputs und batchSize valide sind (nicht leer und kompatibel mit der Layergröße). Werfen Sie eine Exception mit einer aussagekräftigen Fehlermeldung, wenn eine inkompatibilität vorliegt.

Anschließend muss sowohl die cost- als auch die accuracyHistory geleert werden, da diese nur für jeweils einen Trainingslauf gültig sind.

Das training des Neuronalen Netzes passiert, in dem für jeden Input das ganze Netz durch Vorwärtspropagation ausgewertet wird.

Berechnen Sie anschließend auf basis der Loss-Function die Kosten für die Epoche (`epochCost`) und die Genauigkeit (`epochAccuracy`). Anschließend wird das Netz durch Rückwärtspropagation trainiert.

Erinnern Sie sich, dass die Rückwärtspropagation die Werte und Biase nicht bei jedem Aufruf modifiziert, sondern akkumuliert. Die Batchgröße gibt an, nach wie vielen durchlaufenen Inputs die Layer ihre akkumulierten Werte und Biase zurückschreiben.[8].

Updaten Sie als letztes die cost- und accuracyHistory des Netzes.

Die Methode `computeAccuracy` prüft ebenfalls zuerst ob eine Loss-Function existiert und danach ob die Inputs valide sind (nicht leer und kompatibel mit der Layergröße). Anschließend wollen wir berechnen, wie viele der Inputs das Netz korrekt vorhersagt. Werten Sie das Netz dafür auf jedem Input aus und akkumulieren sie den Wert der Loss-Function (`totalCorrect`). Die finale Accuracy ist die der quotient der korrekten vorhersagen aus allen Eingaben.

[8] Das Akkumulieren der Gradienten über mehrere Beispiele (Batches) macht das Training stabiler, da der gemittelte Gradient weniger verrauscht ist und einzelne Ausreißer weniger Gewicht haben. Würden die Gewichte nach jedem einzelnen Beispiel aktualisiert, könnte eine zufällige Häufung untypischer Beispiele das Netz in eine ungünstige Richtung verschieben.

Aufgabe 7: Binary Cross-Entropy Loss (Gesamt: 2 Punkte)

Ziel dieser Aufgabe wird es sein, die bereits in `include/loss/BinaryCrossEntropy.h` deklarierte Binary Cross-Entropy Lossfunktion in `src/lib/loss/BinaryCrossEntropy.cpp` zu implementieren.

7a) getCostValue (1 Punkt)

Implementieren Sie die Methode `getCostValue` der Klasse `BinaryCrossEntropy` wie folgt, wobei n der Länge des Vektors `predictedOutput` entspricht:

$$\vec{P} = \begin{pmatrix} a_0 \\ \vdots \\ a_{n-1} \end{pmatrix} \text{ mit } a_i = log(clampProbability(predictedOutput[i]))$$

$$\vec{Q} = \begin{pmatrix} b_0 \\ \vdots \\ b_{n-1} \end{pmatrix} \text{ mit } b_i = log(1 - clampProbability(predictedOutput[i]))$$

$$\vec{R} = \begin{pmatrix} c_0 \\ \vdots \\ c_{n-1} \end{pmatrix} \text{ mit } c_i = 1 - actualOutput[i]$$

$$getCostValue(predictedOutput, actualOutput) = -\frac{(actualOutput \cdot P + R \cdot Q)}{n}$$

7b) getAccuracyValue and getGradient (1 Punkt)

Implementieren sie die Methode `getAccuracyValue` der Klasse `BinaryCrossEntropy`, welche `predictedOutput` zunächst in einen binären Vektor

$$\vec{V} = \begin{pmatrix} v_0 \\ \vdots \\ v_{n-1} \end{pmatrix} \quad \texttt{mit} \quad v_i = \begin{cases} 1.0 & \text{wenn } predictedOutput[i] > 0.5 \\ 0.0 & \text{sonst} \end{cases}$$

konvertiert, wobei n der Länge des Vektors `predictedOutput` entspricht. Um die Accuracy bestimmen zu können, wird die Anzahl komponentenweiser Übereinstimmungen ($correct$) zwischen V und `actualOutput` bestimmt, sodass schließlich der Rückgabewert von `getAccuracyValue` wie folgt bestimmt werden kann:

$$getAccuracyValue(predictedOutput, actualOutput) = \frac{correct}{n}$$

Implementieren Sie abschließend die Methode `getGradient` der Klasse `BinaryCrossEntropy`, welche den Gradienten g wie folgt berechnen und zurückgeben soll:

$$\vec{G} = \begin{pmatrix} g_0 \\ \vdots \\ g_{n-1} \end{pmatrix} \text{ mit } g_i = -\frac{\left(\frac{actualOutput[i]}{clampProbability(predictedOutput[i])} - \frac{(1 - actualOutput[i])}{1 - clampProbability(predictedOutput[i])}\right)}{n}$$

Aufgabe 8: Memory Leaks (1 Punkt)

Stellen Sie sicher, dass Ihre Implementierung keine Speicherlecks (Memory leaks) beinhaltet.

Hinweis: Sie können dafür Werkzeuge wie Valgrind[9] oder AddressSanitizer[10] nutzen. Machen Sie sich selbst mit den jeweiligen Werkzeugen vertraut. Sie können die Werkzeuge über das Modulsystem des Lichtenbergclusters laden.

[9] https://valgrind.org
[10] https://clang.llvm.org/docs/AddressSanitizer.html

Hinweise zur Abgabe

Hier ein paar zusätzliche Formalitäten zu Ihrer Abgabe. Falls Sie diese nicht beachten, ist es möglich, dass Sie keine Punkte für eine oder alle Aufgaben erhalten.

Angabe der Autorschaft

Geben Sie in der Datei `authors.h` an, wer von Ihnen an welcher Teilaufgabe mitgearbeitet hat. Mehrere Namen pro Teilaufgabe sind ok, trennen Sie diese z.B. mit Komma. Wir fordern eine Teilnahme von allen! Sollten Sie nicht an dem Praktikum mitgearbeitet haben (basierend auf den Angaben), erhalten Sie 0 Punkte auf das ganze Praktikum.

Nameskonvention

Benennen Sie Ihre Abgabe bitte nach dem folgenden Schema: `Px_Gy.zip`, wobei `x` der Nummer des Praktikums und `y` Ihrer Gruppennummer entspricht.

Hochladen des Quellcodes

Zippen Sie folgende Dateien und laden Sie das zip-Archiv in Moodle hoch:

- Den Ordner /include/
- Den Ordner /src/
- Die Datei CMakeLists.txt
- Die Datei authors.h
- Die Datei ai_statement.txt

Sie sind alle für die korrekte Abgabe verantwortlich. Zu spät eingereichte Dateien, Dateien mit fehlenden Lösungen, etc., liegen allein in Ihrer Verantwortung.

Geben Sie in `ai_statement.txt` an, welche IDE Erweiterungen (z.B. Code-Completion Tools) o.Ä. Sie bei der Bearbeitung verwendet haben, und zu welchem Zweck bzw. in welchem Umfang. Das Verwenden von Agenten (z.B. Claude Code o.Ä.) und LLMs (z.B. ChatGPT) zur Lösung der Aufgaben ist nicht zulässig und stellt einen Täuschungsversuch dar!

Die Ausführbarkeit der Abgabe auf dem Lichtenberg-Hochleistungsrechner ist entscheidend, da dieses System zur Bewertung verwendet wird. Sollte Ihre Abgabe auf dem Lichtenberg-Hochleistungsrechner nicht kompilierbar und ausführbar sein, ist mit Punktabzug zu rechnen.

Compiletests

In jedem Praktikum werden Ihnen öffentliche Compiletest für Ihre Abgabe bereitgestellt. Diese werden verwendet, um die Kompilierbarkeit Ihrer Abgabe unabhängig vom Bearbeitungsstand zu überprüfen. Zudem wird in diesen Tests sichergestellt, dass beispielsweise Funktionssignaturen unverändert geblieben, und somit kompatibel mit unserem Testframework sind.

Wichtig: Sie erhalten potenziell keine Punkte für Teilaufgaben mit fehlschlagenden öffentlichen Signaturtests! Sie sind dafür verantwortlich, einen kompilierenden, mit den Vorgaben kompatiblen Code einzureichen. Verändern Sie die vorgebenen Funktionssignaturen nicht!

Hinweis: Nutzen Sie zudem die öffentlichen Tests zur Selbstkontrolle vor Einreichen Ihrer Abgabe.

Wichtig: Sie erhalten **keine** Punkte für das Bestehen der öffentlichen Signaturtests.

Hinweise zum Lichtenberg

Der Lichtenberg-Hochleistungsrechner ist aufgeteilt in sogenannte Loginknoten und Rechenknoten, wobei sich alle das gleiche Dateisystem teilen. Erstere sind mit **ssh** und **scp** zu erreichen und werden benutzt um Rechenaufgaben für letztere zu kreieren. Wenn Sie eine sinnvolle Linux-Distribution verwenden, kann /bin/bash nativ **ssh** und Sie können das Dateisystem des Lichtenberg direkt in Ihrem Explorer (z.B. Nemo) einbinden. Wenn Sie Windows benutzen, empfiehlt sich das Terminal bzw. PuTTy für **ssh** und WinSCP für **scp**.

Sobald Sie eingeloggt sind, sollten Sie zusätzliche Softwarepakete laden. Das erreichen Sie beispielsweise mit **module load git cmake/4.2 gcc/13**. Kopieren Sie die Dateien auf den Hochleistungsrechner und kompilieren Sie dort das Programm.

Wenn Sie eine Rechenaufgabe lösen möchten, müssen Sie dafür SLURM benutzen. Effektiv führen Sie den Befehl **sbatch <filename>** aus, wobei <filename> hier eine Datei ist, die, sobald Ihr Programm Zeit und Resourcen zugeteilt bekommen hat, auf den Rechenknoten ausgeführt wird. Ein beispielhaftes Skript sieht so aus:

```
#!/bin/bash

#SBATCH -J parprog
#SBATCH -e /work/scratch/kurse/kurs00107/<TUID>/stderr/stderr.parprog.%j.txt
#SBATCH -o /work/scratch/kurse/kurs00107/<TUID>/stdout/stdout.parprog.%j.txt
#SBATCH -C avx512
#SBATCH -n 1
#SBATCH --mem-per-cpu=1024
#SBATCH --time=60
#SBATCH --cpus-per-task=8
#SBATCH -c 8
#SBATCH -A kurs00107
#SBATCH -p kurs00107
#SBATCH --reservation=kurs00107

module load git cmake/4.2 gcc/13

echo "This is job $SLURM_JOB_ID"
build/DNN_app
```

Der Kurs ist ab dem 02. November 2026 freigeschaltet. Alle Informationen erhalten Sie auch hier: `https://www.hrz.tu-darmstadt.de/hlr/nutzung_hlr/zugang_hlr/loginknoten_hlr/index.de.jsp`
Wir empfehlen Ihnen das vom HRZ bereitgestellte Videotutorial *Introduction to the Lichtenberg Cluster*[11] anzusehen.

Sollten Sie große Ausgabedaten erzeugen, legen Sie diese bitte in `/work/scratch/kurse/kurs00107/TU-ID` ab, um Quotaüberschreitungen in `/work/kurse/kurs00107` zu vermeiden.
Wichtig: Dateien auf `/work/scratch/kurse/kurs00107/TU-ID` werden 8 Wochen nach der Erstellung gelöscht. Zudem wird von dem Verzeichnis kein Backup angelegt.

Für den Fall, dass Probleme oder Fragen Ihrerseits bezüglich der Nutzung des Lichtenberg-Hochleistungsrechners auftreten, möchten wir Sie auf das FAQ sowie die Website des HRZ (`https://www.hrz.tu-darmstadt.de/hlr/support_hlr/faq_hlr/index.en.jsp`) hinweisen. Selbstverständlich können Sie sich ebenfalls im Rahmen der angebotenen Sprechstunden an die Tutor:innen wenden.

[11] `https://www.hrz.tu-darmstadt.de/hlr/support_hlr/tutorials_hlr/index.de.jsp`